

7교시: 디스크 연동 1 - bind mount로 host 파일 서빙

수업 목표

- host disk directory를 container path에 bind mount하는 구조를 이해한다.
- nginx container가 image 안 파일이 아니라 host 파일을 읽어 응답하는 흐름을 확인한다.
- host 파일 수정이 container 응답에 즉시 반영되는 것을 evidence로 기록한다.

50분 흐름

시간	활동	비중	학생 산출
0-5분	bind mount 개념과 경로 기준 확인	설명 10%	source/destination note
5-15분	실습 파일과 host path 확인	실행 20%	path evidence
15-28분	nginx에 bind mount 연결	실행 25%	run/ps evidence
28-38분	host 파일 수정 후 응답 변경 확인	실행 25%	v1/v2 evidence
38-45분	inspect로 mount 구조 확인	실행 10%	mount evidence
45-50분	cleanup과 named volume 연결	설명 10%	다음 질문

Visual 1: Docker bind mount와 host disk

Week 2 Day 2
Lesson 7

디스크 연동 (Bind Mount) 실습 - nginx

호스트 디렉터리를 컨테이너에 연결하여 파일을 바로 반영합니다.

실행 80%
핵심 실습

Host Disk (호스트 디스크)

~/labs/disk-mount/html

labs

disk-mount

html

index.html

Source (호스트 경로)
\$(pwd)/labs/disk-mount/html
이 디렉터리의 파일이 컨테이너에 연결됨

Bind Mount

읽기 전용 (Read Only)
:ro

Container (컨테이너)

nginx

/usr/share/nginx/html

index.html (Read Only)

Destination (컨테이너 경로)
/usr/share/nginx/html
nginx가 이 경로를 웹 루트로 사용

포트 매핑
18083:80

웹 브라우저 (접속 확인)

http://localhost:18083

nginx

Disk Bind Mount 실습

버전: v1

응답 변경
호스트 파일 수정 시 즉시 반영됨

실행 명령

```
docker run -d --name nginx-disk -p 18083:80 \
-v $(pwd)/labs/disk-mount/html:/usr/share/nginx/html:ro nginx:alpine
```

-v source:destination:ro

Source: \$(pwd)/labs/disk-mount/html

Destination: /usr/share/nginx/html

:ro (읽기 전용)

주의사항

- Source 경로는 절대 경로를 권장합니다.
- Destination 경로를 정확히 지정해야 nginx가 파일을 찾습니다.
- Read Only(:ro)로 마운트하면 컨테이너에서 파일을 변경할 수 없습니다.
- 컨테이너 내부에서 파일 쓰기를 시도하면 실패합니다.

1 초기 상태 (v1)

index.html (호스트)
<h1>Disk Bind Mount</h1>
<p>버전: v1</p>

접속

브라우저 응답

Disk Bind Mount
버전: v1

2 호스트 파일 수정

index.html 수정
<h1>Disk Bind Mount</h1>
<p>버전: v2 (수정됨)</p>

파일 저장

3 즉시 반영 (v2)

접속

브라우저 응답

Disk Bind Mount
버전: v2 (수정됨)

응답 변경 확인!
nginx가 호스트 파일을 실시간으로 제공

실습 체크리스트

- ✓ 컨테이너 실행 (포트 18083 접근)
- ✓ 호스트 파일 수정 후 응답 변경 확인
- ✓ 로그 확인
- ✓ 정리 (stop / rm)

핵심 포인트: Bind Mount를 사용하면 호스트 디스크의 변경이 컨테이너에 즉시 반영됩니다.

증거 기록: 명령, 결과(스크린샷), 변경 전/후 비교를 기록하세요.

이 이미지는 host의 `labs/disk-mount/html` directory가 container의 `/usr/share/nginx/html` 에 연결되고, nginx가 그 파일을 browser에 제공하는 흐름을 보여준다. `:ro` 는 container가 mount된 파일을 읽기 전용으로 본다는 뜻이다.

실습 파일

시작 파일은 [labs/disk-mount/html/index.html](#)에 있다.

```
<h1>Disk mount lab - host file v1</h1>
<p>이 파일은 host disk에 있고 nginx container가 bind mount로 읽는다.</p>
```

storage model 설명

Docker storage는 크게 세 가지를 구분해야 한다. 첫째, image layer는 build 결과이며 read-only로 취급한다. 둘째, container writable layer는 container가 실행되는 동안 생기는 임시 변경 공간이다. 셋째, bind mount와 volume은 container lifecycle 바깥에 데이터를 두는 방법이다.

bind mount는 host filesystem의 특정 path를 container path에 연결한다. 개발 중 source code나 설정 파일을 container에 빠르게 반영하기 좋다. 하지만 host path에 강하게 의존하므로 다른 사람의 Mac, Windows, Linux에서 같은 경로가 아닐 수 있다. 그래서 배포용 artifact로는 Dockerfile build가 더 적합하고, 개발용 빠른 확인에는 bind mount가 유용하다.

mount overlay 주의

Docker 공식 문서는 container 안에 기존 파일이 있는 directory 위로 bind mount를 걸면 기존 내용이 mount된 host directory에 가려질 수 있다고 설명한다. nginx image 안의 `/usr/share/nginx/html`에는 기본 index 파일이 있지만, 우리가 host directory를 그 위치에 mount하면 host directory 내용이 보인다. 이것은 오류가 아니라 mount의 동작 방식이다.

Hands-on 1: bind mount 실행

repository root에서 실행한다.

```
docker run -d \
  --name paperclip-day2-disk \
  -p 18083:80 \
  -v "$PWD/week2/day2/labs/disk-mount/html:/usr/share/nginx/html:ro" \
  nginx:1.27-alpine

docker ps --filter name=paperclip-day2-disk
curl -s http://localhost:18083
docker exec paperclip-day2-disk ls -l /usr/share/nginx/html
```

macOS/Linux shell에서는 위치를 `$PWD`를 쓴다. 경로 오류가 나면 먼저 `pwd`가 repository root인지 확인한다.

전체 디스크 연동 절차는 [hands-on-lab.md](#)의 Phase F를 따른다. 이 교시에서는 source path, destination path, mode, inspect evidence를 반드시 기록한다.

Linux 사전 테스트 결과: v1

```
curl -s http://localhost:18083
```

 핵심 출력:

```
<h1>Disk mount lab - host file v1</h1>
<p>이 파일은 host disk에 있고 nginx container가 bind mount로 읽는다.</p>
```

내부 파일 확인:

```
total 0
-rwxrwxrwx  1 1000    1000      279 index.html
```

Hands-on 2: host 파일 수정 후 응답 변경

[labs/disk-mount/html/index.html](#)을 다음처럼 바꾼다.

```
<h1>Disk mount lab - host file v2</h1>
<p>host disk 파일을 수정하자 container 응답이 바뀐다.</p>
```

다시 확인한다.

```
curl -s http://localhost:18083
```

Linux 사전 테스트 결과: v2

```
<h1>Disk mount lab - host file v2</h1>
<p>host disk 파일을 수정하자 container 응답이 바뀐다.</p>
```

이 결과는 image rebuild 없이 host disk의 파일 변경이 container 응답에 반영되는 것을 보여준다. 개발 환경에서는 유용하지만, 배포용 image를 만들 때는 Dockerfile build로 파일을 image에 포함하는 기준이 필요하다.

Hands-on 3: mount 구조 확인

```
docker inspect paperclip-day2-disk
```

Linux 사전 테스트 핵심 출력

```
"Mounts": [
  {
    "Type": "bind",
    "Source": "/mnt/d/paperclip/week2/day2/labs/disk-mount/html",
    "Destination": "/usr/share/nginx/html",
    "Mode": "ro",
    "RW": false
  }
]
```

cleanup

```
docker stop paperclip-day2-disk
docker rm paperclip-day2-disk
```

bind mount 판단표

항목	의미	확인 질문
Source	host disk path	실제 존재하는 directory인가
Destination	container 안 path	app이 읽는 위치와 맞는가
:ro	read only mount	container에서 수정할 필요가 없는가
path error	mount source 없음/오타	pwd, ls로 확인했는가
운영 위험	host와 container 경계가 섞임	배포용 image와 개발용 mount를 구분했는가

장애 drill

증상	가능한 원인	확인
nginx 기본 화면이 계속 보임	mount destination이 틀림	docker inspect, Mounts 확인
v2가 안 보임	browser cache 또는 파일 저장 안 됨	curl -s, host file 저장 확인
container가 바로 종료	mount path 문제보다는 nginx/process 문제 가능	docker logs
permission denied	host file permission 또는 Desktop file sharing	ls -l, Docker Desktop file sharing

추가 실습: read-only 확인

:ro 로 mount했기 때문에 container 안에서 파일을 수정하려 하면 실패해야 한다.

```
docker exec paperclip-day2-disk sh -c "echo should-fail > /usr/share/nginx/html/from-container.txt"
```

예상:

- read-only file system 또는 permission 관련 error가 나온다.

운영 의미:

- 개발 중에도 container가 host source를 임의로 바꾸지 못하게 할 수 있다.
- read/write가 필요한 mount와 read-only mount를 구분해야 한다.

평가 기준

기준	2점 evidence
bind mount 실행	source/destination/port를 정확히 기록했다.
v1/v2 확인	host file 수정 전후 응답 차이를 확인했다.
inspect	Type: bind, Source, Destination, RW: false를 찾았다.
cleanup	container를 stop/rm으로 정리했다.

공식 근거 링크

- Docker Docs: Bind mounts, <https://docs.docker.com/engine/storage/bind-mounts/>
- Docker Docs: docker inspect, <https://docs.docker.com/reference/cli/docker/inspect/>
- Docker Docs: Storage, <https://docs.docker.com/engine/storage/>